

Linked List Templates

Four patterns cover almost every linked list problem.

Consistent naming throughout: `sentinel` , `previous` , `current` , `next` , `slow` , `fast` .

Quick Reference Table

#	Name	Description	Intuition	Variation
203	Remove Linked List Elements	Remove all nodes whose value equals <code>val</code> .		Standard
83	Remove Duplicates from Sorted List	Keep exactly one copy of each value. List is sorted.		Standard
82	Remove Duplicates from Sorted List II	Remove ALL nodes that have duplicate values — only distinct-valued nodes remain.		Standard
19	Remove Nth Node From End of List	Remove the nth node from the end. Return the head.		Standard
206	Reverse Linked List	Reverse the entire linked list. Return the new head.	flip each node's arrow to point at the node you just came from; <code>previous</code> is the reversed list built so far.	Standard
92	Reverse Linked List II	Reverse only the nodes from position <code>left</code> to <code>right</code> (1-indexed).		Standard
25	Reverse Nodes in k-Group	Reverse every consecutive group of k nodes. Leave remaining nodes (< k) as-is.		Standard
876	Middle of Linked List	Return the middle node. For even length, return the second middle.		Standard
141	Linked List Cycle	Return true if the list contains a cycle.		Standard
142	Linked List Cycle II	Return the node where the cycle begins. Return null if no cycle.	the distance from head to the cycle entry equals the distance from the meeting point to the entry, so two 1-step walkers from those spots collide exactly at the entry.	Standard
21	Merge Two Sorted Lists	Merge two sorted linked lists into one sorted list.	like a zipper — always splice on whichever current head is smaller, then advance that list.	Standard
23	Merge K Sorted Lists	Merge k sorted linked lists into one sorted list.		Standard
143	Reorder List	Reorder list as $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$		Standard
388	Odd Even Linked List	Group all odd-indexed nodes first, then even-indexed nodes. Preserve relative order within each group. (Indexing is 1-based.)		Standard
2	Add Two Numbers	Two non-empty linked lists represent non-negative integers stored in reverse order. Add and return the sum as a linked list in reverse order.		loop until carry clears
88	Merge Sorted Array	Merge <code>nums2</code> into <code>nums1</code> in-place. <code>nums1</code> has enough space at the end. Both arrays are sorted.		three pointers from END
234	Palindrome Linked List	Check if a singly linked list is a palindrome in $O(n)$ time and $O(1)$ space.		in-place reverse
160	Intersection of Two Linked Lists	Find the node at which two singly linked lists intersect. Return null if they do not intersect.		redirect to other list on null
24	Swap Nodes in Pairs	Swap every two adjacent nodes in a linked list and return the modified head.	A sentinel removes head special-casing. For each pair, <code>previous</code> is the node before the pair; re-wire the three pointers so the second node leads, then advance <code>previous</code> two nodes forward.	Standard
61	Rotate List	Rotate a linked list to the right by <code>k</code> places.	Rotating right by <code>k</code> is equivalent to making the list circular, then breaking it <code>k % length</code> nodes before the old tail. Compute the length, close the ring, then walk to the new tail.	only the remainder matters
86	Partition List	Partition a linked list so all nodes with value <code>< x</code> come before nodes with value <code>>= x</code> . Preserve the relative order within each group.	Build two separate chains with two sentinels — one for the "less" group and one for the "greater-or-equal" group — then stitch them together. Preserving original order falls out naturally because nodes are appended in traversal order.	Standard
109	Convert Sorted List to Binary Search Tree	Convert a sorted linked list to a height-balanced BST.	A sorted list is an in-order traversal of a BST. Repeatedly pick the middle node as the subtree root so each side gets a balanced number of nodes; recurse on the left half (before middle) and right half (after middle).	stop at exclusive tail

#	Name	Description	Intuition	Variation
1 3 8	Copy List with Random Pointer	Deep-copy a linked list where each node has a <code>random</code> pointer to any node or null.	Interleave each clone directly after its original so a clone's <code>random</code> is reachable as <code>original.random.next</code> — no hash map needed. Three passes: weave clones in, wire random pointers, then unweave.	Standard
1 4 8	Sort List	Sort a linked list. Aim for $O(n \log n)$ time and $O(1)$ extra space (top-down recursion uses $O(\log n)$ stack).	Merge sort fits linked lists perfectly — splitting is a pointer cut and merging reuses the #21 zipper. Find the middle, sort each half, merge.	Standard
2 3 7	Delete Node in a Linked List	Delete a node from a singly linked list given only a reference to that node (not the head). The node is guaranteed not to be the tail.	Without access to the previous node you cannot unlink the given node directly, but you can copy the next node's value into it and then delete the next node instead.	copy successor's value
3 6 9	Plus One Linked List	A non-negative integer is stored as a linked list of digits in big-endian order (most significant first). Add one to it and return the resulting list.	Reverse the list so addition flows least-significant-first like normal carry arithmetic, add one with carry propagation, then reverse back. A possible new leading digit is handled by extending the tail after reversal.	seed carry with the +1
4 3 0	Flatten a Multilevel Doubly Linked List	Flatten a multilevel doubly linked list where nodes may have a <code>child</code> list. Produce a single-level list using the <code>next</code> pointers; all <code>child</code> pointers must be null.	A child list should be spliced in immediately after its parent, before the parent's original <code>next</code> . A stack lets you remember each deferred <code>next</code> while you dive into a child branch — effectively an iterative DFS.	clear child after splicing
4 4 5	Add Two Numbers II	Two numbers are stored as linked lists in big-endian order (most significant digit first). Add them and return the sum as a linked list, also in big-endian order.	Addition needs least-significant digits first, but reversing the inputs is discouraged here. Push all digits onto two stacks; popping yields least-significant-first. Build the result by prepending nodes so the final list is big-endian.	prepend to keep big-endian order
7 0 8	Insert into a Sorted Circular Linked List	Insert a value into a sorted circular linked list and return a reference to any node in the list. The given pointer may reference any node, or be null for an empty list.	Walk one full loop looking for the correct gap between a node and its successor. Three cases cover it: a normal in-between slot, the wrap-around boundary (max→min) where the value is an extreme, and a uniform-value list where any spot works.	wrap-around boundary
7 2 5	Split Linked List in Parts	Split a linked list into k consecutive parts as equal in size as possible. Earlier parts have sizes greater than or equal to later parts; some trailing parts may be empty.	With length n , each part gets n / k nodes, and the first $n \% k$ parts get one extra. Walk through cutting off each part at its computed size.	first remainder parts get +1
1 1 7 1	Remove Zero Sum Consecutive Nodes from Linked List	Repeatedly remove consecutive sequences of nodes that sum to zero until no such sequence remains. Return the resulting list.	Use prefix sums. If two nodes share the same running prefix sum, every node strictly between them sums to zero and can be dropped. A hash map from prefix sum to node lets you splice out those runs in one pass.	keep the latest node per sum
1 2 6 5	Print Immutable Linked List in Reverse	Print all values of an immutable linked list in reverse, using only the exposed <code>getNext()</code> and <code>printValue()</code> API, without modifying the list.	Recursion naturally reverses order — recurse to the end first, then print on the way back up the call stack.	recurse first, print on unwind

When to Use — Signal → Pattern

Signal in the prompt	Pattern
"remove / delete / skip" nodes by value or duplicate	Delete / Filter (sentinel + <code>previous</code>)
"reverse" the list, a sub-range, or in k-groups	Reversal
"middle node", "cycle", "nth node from the end"	Fast / Slow
"merge two sorted lists" (or k lists)	Merge (sentinel + heap for k)
chained steps ("reorder", "palindrome", "sort")	Composite (combine the above)

All Four Templates

```
// 1. DELETE / FILTER – sentinel guards head removal; previous tracks last kept node
// MENTAL MODEL: previous always points at the last node you decided to keep, so re-wiring it skips anything unwanted.
// WHEN: "remove / delete / skip nodes by value or duplicate"
ListNode sentinel = new ListNode(0);
sentinel.next = head;
ListNode previous = sentinel, current = head;
while (current != null) {
    if (shouldDelete(current)) {
        previous.next = current.next;    // skip node
    } else {
        previous = current;              // keep node, advance previous
    }
    current = current.next;
}
return sentinel.next;

// 2. REVERSAL – re-wire one node at a time
// MENTAL MODEL: flip each arrow to point backward; previous is the growing reversed list trailing behind current.
// WHEN: "reverse the list (whole, partial, or in k-groups)"
ListNode previous = null, current = head;
while (current != null) {
    ListNode next = current.next;    // save before overwrite
    current.next = previous;          // reverse pointer
    previous = current;              // advance
    current = next;
}
return previous;    // new head

// 3. FAST / SLOW – two pointers at different speeds
// MENTAL MODEL: fast covers twice the distance, so when it hits the end slow is exactly halfway; in a loop they must collide.
// WHEN: "middle node", "detect/locate cycle", "nth from end"
ListNode slow = head, fast = head;
while (fast != null && fast.next != null) {
    slow = slow.next;
    fast = fast.next.next;
}
// slow is at middle (or cycle detection: slow == fast → cycle found)

// 4. MERGE – sentinel collects nodes from two lists in order
// MENTAL MODEL: repeatedly splice the smaller head onto a growing result tail, like a zipper.
// WHEN: "merge two (or k) sorted lists"
ListNode sentinel = new ListNode(0);
ListNode current = sentinel;
while (a != null && b != null) {
    if (a.val <= b.val) {
        current.next = a;
        a = a.next;
    } else {
        current.next = b;
        b = b.next;
    }
    current = current.next;
}
current.next = (a != null) ? a : b;
return sentinel.next;
```

Pattern 1 — Delete / Filter

`previous` stays on the last **kept** node. When deleting, re-wire `previous.next` to skip `current`. When keeping, advance `previous` to `current`. Always advance `current`.

#203 Remove Linked List Elements

Description: Remove all nodes whose value equals `val`.

Delete condition: `current.val == val`

```
class Solution {
    public ListNode removeElements(ListNode head, int val) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (current.val == val) {
                previous.next = current.next;
            } else {
                previous = current;
            }
            current = current.next;
        }
        return sentinel.next;
    }
}
```

#83 Remove Duplicates from Sorted List

Description: Keep exactly one copy of each value. List is sorted.

Delete condition: `previous != sentinel && previous.val == current.val`

```
class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (previous != sentinel && previous.val == current.val) {
                previous.next = current.next; // skip duplicate
            } else {
                previous = current;
            }
            current = current.next;
        }
        return sentinel.next;
    }
}
```

#82 Remove Duplicates from Sorted List II

Description: Remove ALL nodes that have duplicate values — only distinct-valued nodes remain.

Key difference from #83: when a duplicate run is detected, skip the entire run (including the first occurrence). Inner `while` advances `current` past the whole run; `previous.next` is rewired to skip all of them.

```

class Solution {
    public ListNode deleteDuplicates(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel, current = head;
        while (current != null) {
            if (current.next != null && current.val == current.next.val) {
                int dupVal = current.val;
                while (current != null && current.val == dupVal)
                    current = current.next; // advance past entire run
                previous.next = current; // skip all duplicates
            } else {
                previous = current;
                current = current.next;
            }
        }
        return sentinel.next;
    }
}

```

#19 Remove Nth Node From End of List

Description: Remove the nth node from the end. Return the head.

Key: `fast` starts n+1 steps ahead of `slow` (both from sentinel). When `fast == null`, `slow` is just before the target node.

```

class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode fast = sentinel, slow = sentinel;
        for (int i = 0; i <= n; i++) fast = fast.next; // n+1 steps ahead
        while (fast != null) { slow = slow.next; fast = fast.next; }
        slow.next = slow.next.next; // delete target
        return sentinel.next;
    }
}

```

Pattern 2 — Reversal

#206 Reverse Linked List

Description: Reverse the entire linked list. Return the new head.

Key: re-wire one node per iteration. `previous` trails behind as the new "next" for each node.

Intuition: flip each node's arrow to point at the node you just came from; `previous` is the reversed list built so far.

```

class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode previous = null, current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        return previous;
    }
}

```

#92 Reverse Linked List II

Description: Reverse only the nodes from position `left` to `right` (1-indexed).

Key: walk `previous` to the node just before `left`. Then insert-at-front (`right - left`) times — each time take `current.next`, unlink it, and attach it right after `previous`.

```
class Solution {
    public ListNode reverseBetween(ListNode head, int left, int right) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel;
        for (int i = 0; i < left - 1; i++) previous = previous.next;
        ListNode current = previous.next;
        for (int i = 0; i < right - left; i++) {
            ListNode next = current.next;
            current.next = next.next;    // unlink next
            next.next = previous.next;   // attach next at front of reversed region
            previous.next = next;
        }
        return sentinel.next;
    }
}
```

#25 Reverse Nodes in k-Group

Description: Reverse every consecutive group of `k` nodes. Leave remaining nodes (`< k`) as-is.

Key: `groupPrev` is the tail of the already-processed part. Find the `k`th node ahead; if it exists, reverse the group using the same insert-at-front technique as #92.

```
class Solution {
    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode groupPrev = sentinel;
        while (true) {
            ListNode kth = getKth(groupPrev, k);
            if (kth == null) break;
            ListNode groupNext = kth.next;
            ListNode previous = groupNext, current = groupPrev.next;
            while (current != groupNext) {
                ListNode next = current.next;
                current.next = previous;
                previous = current;
                current = next;
            }
            ListNode tmp = groupPrev.next;    // will become new tail of this group
            groupPrev.next = kth;             // kth is new head of reversed group
            groupPrev = tmp;
        }
        return sentinel.next;
    }
    private ListNode getKth(ListNode node, int k) {
        while (node != null && k-- > 0) node = node.next;
        return node;
    }
}
```

Pattern 3 — Fast / Slow Pointers

`fast` moves 2x per step. When `fast` reaches the end, `slow` is at the middle.
When `slow == fast` after the start, a cycle is detected.

#876 Middle of Linked List

Description: Return the middle node. For even length, return the second middle.

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

#141 Linked List Cycle

Description: Return true if the list contains a cycle.

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

#142 Linked List Cycle II

Description: Return the node where the cycle begins. Return null if no cycle.

Key: after slow meets fast inside the cycle, reset `slow` to `head`. Both then move 1 step at a time — they meet at the cycle entry.

Intuition: the distance from head to the cycle entry equals the distance from the meeting point to the entry, so two 1-step walkers from those spots collide exactly at the entry.

```
class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) {
                slow = head;                // reset slow to head
                while (slow != fast) {
                    slow = slow.next;
                    fast = fast.next;        // both move 1 step
                }
                return slow;                // cycle entry
            }
        }
        return null;
    }
}
```


Pattern 4 — Merge

`sentinel` collects the merged result. `current` is the write pointer.

#21 Merge Two Sorted Lists

Description: Merge two sorted linked lists into one sorted list.

Intuition: like a zipper — always splice on whichever current head is smaller, then advance that list.

```
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                current.next = list1;
                list1 = list1.next;
            } else {
                current.next = list2;
                list2 = list2.next;
            }
            current = current.next;
        }
        current.next = (list1 != null) ? list1 : list2;
        return sentinel.next;
    }
}
```

#23 Merge K Sorted Lists

Description: Merge k sorted linked lists into one sorted list.

Key: min-heap of size $\leq k$. Always extract the smallest current head; push its next node back.

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode node : lists) if (node != null) pq.offer(node);
        while (!pq.isEmpty()) {
            current.next = pq.poll();
            current = current.next;
            if (current.next != null) pq.offer(current.next);
        }
        return sentinel.next;
    }
}
```

Pattern 5 — Composite (Multi-step)

Problems that chain multiple patterns together.

#143 Reorder List

Description: Reorder list as $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$

Steps: (1) find middle with slow/fast, (2) reverse second half, (3) merge the two halves.

```

class Solution {
    public void reorderList(ListNode head) {
        // Step 1: find middle
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // Step 2: reverse second half
        ListNode second = slow.next;
        slow.next = null;
        ListNode previous = null, current = second;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        // Step 3: merge first half and reversed second half
        ListNode first = head;
        second = previous;
        while (second != null) {
            ListNode nextFirst = first.next, nextSecond = second.next;
            first.next = second;
            second.next = nextFirst;
            first = nextFirst;
            second = nextSecond;
        }
    }
}

```

#328 Odd Even Linked List

Description: Group all odd-indexed nodes first, then even-indexed nodes. Preserve relative order within each group. (Indexing is 1-based.)

```

class Solution {
    public ListNode oddEvenList(ListNode head) {
        if (head == null) return null;
        ListNode odd = head, even = head.next, evenHead = even;
        while (even != null && even.next != null) {
            odd.next = even.next;    odd = odd.next;
            even.next = odd.next;    even = even.next;
        }
        odd.next = evenHead;
        return head;
    }
}

```

Pattern Summary

Pattern	Sentinel	Naming	When to use
Delete / Filter	Yes	<code>previous</code> , <code>current</code>	Remove or skip nodes by condition
Reversal	No (Yes for partial)	<code>previous</code> , <code>current</code> , <code>next</code>	Re-wire pointer direction
Fast / Slow	No	<code>slow</code> , <code>fast</code>	Middle, cycle, nth from end
Merge	Yes	<code>current</code>	Combine two or more lists
Composite	Both	Mix of above	Reorder, sort, rearrange

Sentinel Cheat Sheet

SENTINEL: dummy node before head so head removal needs no special-casing; use it whenever the head itself might be removed/replaced.

```
// Always use sentinel when the head itself might be removed or replaced:
ListNode sentinel = new ListNode(0);
sentinel.next = head;
// ... operations ...
return sentinel.next; // head may have changed; sentinel.next is always correct
```

82 vs 83 — Side by Side

	#83 (keep one)	#82 (remove all)
Delete condition:	<code>previous.val == current.val</code>	<code>current.val == current.next.val</code>
Action on match:	skip current only	inner while to skip entire run
previous advances?:	only when keeping	only when not a dup

#2 Add Two Numbers

Description: Two non-empty linked lists represent non-negative integers stored in reverse order. Add and return the sum as a linked list in reverse order.

Algorithm: Use sentinel + carry variable. Traverse both lists simultaneously; at each step compute `sum = l1.val + l2.val + carry`. Create a new node with `sum % 10`, carry `sum / 10` forward. Continue while either list has nodes or carry is non-zero.

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) { // ← VARIATION: loop until carry clears
            int sum = carry;
            if (l1 != null) { sum += l1.val; l1 = l1.next; }
            if (l2 != null) { sum += l2.val; l2 = l2.next; }
            carry = sum / 10; // ← VARIATION: propagate carry
            current.next = new ListNode(sum % 10);
            current = current.next;
        }
        return sentinel.next;
    }
}
```

Time $O(\max(m, n))$ | **Space** $O(\max(m, n))$

#88 Merge Sorted Array

Description: Merge `nums2` into `nums1` in-place. `nums1` has enough space at the end. Both arrays are sorted.

Algorithm: Fill from the END backwards using three pointers `i` (end of `nums1` values), `j` (end of `nums2`), `k` (end of merged array). This avoids overwriting unprocessed elements in `nums1`.

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1; // ← VARIATION: three pointers from END
        while (i >= 0 && j >= 0)
            nums1[k--] = (nums1[i] >= nums2[j]) ? nums1[i--] : nums2[j--];
        while (j >= 0) nums1[k--] = nums2[j--]; // ← copy remaining nums2
    }
}
```

Time $O(m + n)$ | **Space** $O(1)$

#234 Palindrome Linked List

Description: Check if a singly linked list is a palindrome in $O(n)$ time and $O(1)$ space.

Algorithm: Composite — (1) fast/slow to find middle, (2) reverse the second half in-place, (3) compare both halves. Variation: combines three separate patterns.

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        // Step 1: find middle using fast/slow
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        // Step 2: reverse second half          // ← VARIATION: in-place reverse
        ListNode previous = null, current = slow;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        // Step 3: compare halves
        ListNode i = head, j = previous;
        while (j != null) {
            if (i.val != j.val) return false;
            i = i.next;
            j = j.next;
        }
        return true;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#160 Intersection of Two Linked Lists

Description: Find the node at which two singly linked lists intersect. Return null if they do not intersect.

Algorithm: Two pointers `a` and `b` start at `headA` and `headB`. When either reaches the end, redirect it to the other list's head. After at most $m + n$ steps they meet at the intersection (or both become null for no intersection). This works because both pointers travel the same total distance.

```
class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode a = headA, b = headB;
        while (a != b) {
            a = (a == null) ? headB : a.next;    // ← VARIATION: redirect to other list on null
            b = (b == null) ? headA : b.next;
        }
        return a;    // either the intersection node, or null
    }
}
```

Time $O(m + n)$ | **Space** $O(1)$

Additional Reference Problems

#24 Swap Nodes in Pairs

Description: Swap every two adjacent nodes in a linked list and return the modified head.

Intuition: A sentinel removes head special-casing. For each pair, `previous` is the node before the pair; re-wire the three pointers so the second node leads, then advance `previous` two nodes forward.

Algorithm: Loop while there are at least two nodes ahead of `previous`. Name them `first` and `second`. Splice so order becomes `second` → `first`, reconnect `first.next` to whatever followed `second`, then move `previous` to `first` (now the tail of the swapped pair).

```
class Solution {
    public ListNode swapPairs(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        ListNode previous = sentinel;
        while (previous.next != null && previous.next.next != null) {
            ListNode first = previous.next;
            ListNode second = first.next;
            first.next = second.next;    // first now points past the pair
            second.next = first;        // second leads the pair
            previous.next = second;      // link prior node to new pair head
            previous = first;           // advance to tail of swapped pair
        }
        return sentinel.next;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#61 Rotate List

Description: Rotate a linked list to the right by `k` places.

Intuition: Rotating right by `k` is equivalent to making the list circular, then breaking it `k % length` nodes before the old tail. Compute the length, close the ring, then walk to the new tail.

Algorithm: Measure length and find the tail. Connect tail to head to form a cycle. The new tail sits `length - (k % length)` steps from the head; the node after it is the new head. Break the cycle there.

```
class Solution {
    public ListNode rotateRight(ListNode head, int k) {
        if (head == null || head.next == null || k == 0) {
            return head;
        }
        int length = 1;
        ListNode tail = head;
        while (tail.next != null) {
            tail = tail.next;
            length++;
        }
        k = k % length;                // ← VARIATION: only the remainder matters
        if (k == 0) {
            return head;
        }
        tail.next = head;             // close into a ring
        int stepsToNewTail = length - k;
        ListNode current = head;
        for (int i = 1; i < stepsToNewTail; i++) {
            current = current.next;
        }
        ListNode newHead = current.next;
        current.next = null;          // break the ring
        return newHead;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#86 Partition List

Description: Partition a linked list so all nodes with value $< x$ come before nodes with value $\geq x$. Preserve the relative order within each group.

Intuition: Build two separate chains with two sentinels — one for the "less" group and one for the "greater-or-equal" group — then stitch them together. Preserving original order falls out naturally because nodes are appended in traversal order.

Algorithm: Walk `current` over the list, appending each node to the `less` tail or `greater` tail by comparing with `x`. Terminate the `greater` chain, then connect the `less` tail to the head of the `greater` chain.

```
class Solution {
    public ListNode partition(ListNode head, int x) {
        ListNode lessSentinel = new ListNode(0);
        ListNode greaterSentinel = new ListNode(0);
        ListNode less = lessSentinel, greater = greaterSentinel;
        ListNode current = head;
        while (current != null) {
            if (current.val < x) {
                less.next = current;
                less = less.next;
            } else {
                greater.next = current;
                greater = greater.next;
            }
            current = current.next;
        }
        greater.next = null; // terminate second chain
        less.next = greaterSentinel.next; // splice less → greater
        return lessSentinel.next;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#109 Convert Sorted List to Binary Search Tree

Description: Convert a sorted linked list to a height-balanced BST.

Intuition: A sorted list is an in-order traversal of a BST. Repeatedly pick the middle node as the subtree root so each side gets a balanced number of nodes; recurse on the left half (before middle) and right half (after middle).

Algorithm: Use fast/slow to find the middle of the current range `[head, tail]`. The middle becomes the root. Recurse on `[head, middle)` for the left child and `[middle.next, tail]` for the right child. The half-open range avoids cutting the list and keeps recursion clean.

```
class Solution {
    public TreeNode sortedListToBST(ListNode head) {
        return build(head, null);
    }
    private TreeNode build(ListNode head, ListNode tail) {
        if (head == tail) {
            return null;
        }
        ListNode slow = head, fast = head;
        while (fast != tail && fast.next != tail) { // ← VARIATION: stop at exclusive tail
            slow = slow.next;
            fast = fast.next.next;
        }
        TreeNode root = new TreeNode(slow.val);
        root.left = build(head, slow);
        root.right = build(slow.next, tail);
        return root;
    }
}
```

Time $O(n \log n)$ | **Space** $O(\log n)$

#138 Copy List with Random Pointer

Description: Deep-copy a linked list where each node has a `random` pointer to any node or null.

Intuition: Interleave each clone directly after its original so a clone's `random` is reachable as `original.random.next` — no hash map needed. Three passes: weave clones in, wire random pointers, then unweave.

Algorithm: Pass 1: insert `clone` right after each `current`. Pass 2: set `current.next.random = current.random.next` (guarding null). Pass 3: detach the cloned chain and restore the original list.

```
class Solution {
    public Node copyRandomList(Node head) {
        if (head == null) {
            return null;
        }
        // Pass 1: weave clones into the original list
        Node current = head;
        while (current != null) {
            Node clone = new Node(current.val);
            clone.next = current.next;
            current.next = clone;
            current = clone.next;
        }
        // Pass 2: assign random pointers on the clones
        current = head;
        while (current != null) {
            if (current.random != null) {
                current.next.random = current.random.next;
            }
            current = current.next.next;
        }
        // Pass 3: unweave into two separate lists
        Node sentinel = new Node(0);
        Node copy = sentinel;
        current = head;
        while (current != null) {
            copy.next = current.next;
            copy = copy.next;
            current.next = copy.next; // restore original
            current = current.next;
        }
        return sentinel.next;
    }
}
```

Time $O(n)$ | **Space** $O(1)$

#148 Sort List

Description: Sort a linked list. Aim for $O(n \log n)$ time and $O(1)$ extra space (top-down recursion uses $O(\log n)$ stack).

Intuition: Merge sort fits linked lists perfectly — splitting is a pointer cut and merging reuses the #21 zipper. Find the middle, sort each half, merge.

Algorithm: Use fast/slow (with `fast = head.next` so the left half is never empty) to split the list in two. Recursively sort each half, then merge them with the standard sentinel-based merge.

```

class Solution {
    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }
        ListNode slow = head, fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode second = slow.next;
        slow.next = null;           // cut into two halves
        ListNode left = sortList(head);
        ListNode right = sortList(second);
        return merge(left, right);
    }
    private ListNode merge(ListNode a, ListNode b) {
        ListNode sentinel = new ListNode(0);
        ListNode current = sentinel;
        while (a != null && b != null) {
            if (a.val <= b.val) {
                current.next = a;
                a = a.next;
            } else {
                current.next = b;
                b = b.next;
            }
            current = current.next;
        }
        current.next = (a != null) ? a : b;
        return sentinel.next;
    }
}

```

Time $O(n \log n)$ | **Space** $O(\log n)$

#237 Delete Node in a Linked List

Description: Delete a node from a singly linked list given only a reference to that node (not the head). The node is guaranteed not to be the tail.

Intuition: Without access to the previous node you cannot unlink the given node directly, but you can copy the next node's value into it and then delete the next node instead.

Algorithm: Overwrite `node.val` with `node.next.val`, then bypass `node.next` by setting `node.next = node.next.next`.

```

class Solution {
    public void deleteNode(ListNode node) {
        node.val = node.next.val;    // ← VARIATION: copy successor's value
        node.next = node.next.next;  // then unlink the successor
    }
}

```

Time $O(1)$ | **Space** $O(1)$

#369 Plus One Linked List

Description: A non-negative integer is stored as a linked list of digits in big-endian order (most significant first). Add one to it and return the resulting list.

Intuition: Reverse the list so addition flows least-significant-first like normal carry arithmetic, add one with carry propagation, then reverse back. A possible new leading digit is handled by extending the tail after reversal.

Algorithm: Reverse the list. Walk it adding `carry` (starting at 1); each node becomes `(val + carry) % 10`, carry becomes `(val + carry) / 10`. If carry remains after the last node, append a new node. Reverse back to big-endian.


```

class Solution {
    public ListNode plusOne(ListNode head) {
        ListNode reversed = reverse(head);
        int carry = 1; // ← VARIATION: seed carry with the +1
        ListNode current = reversed, last = reversed;
        while (current != null && carry != 0) {
            int sum = current.val + carry;
            current.val = sum % 10;
            carry = sum / 10;
            last = current;
            current = current.next;
        }
        if (carry != 0) {
            last.next = new ListNode(carry); // extend for a new leading digit
        }
        return reverse(reversed);
    }
    private ListNode reverse(ListNode head) {
        ListNode previous = null, current = head;
        while (current != null) {
            ListNode next = current.next;
            current.next = previous;
            previous = current;
            current = next;
        }
        return previous;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#430 Flatten a Multilevel Doubly Linked List

Description: Flatten a multilevel doubly linked list where nodes may have a `child` list. Produce a single-level list using the `next` pointers; all `child` pointers must be null.

Intuition: A child list should be spliced in immediately after its parent, before the parent's original `next`. A stack lets you remember each deferred `next` while you dive into a child branch — effectively an iterative DFS.

Algorithm: Walk `current` with a stack. When `current` has a child, push `current.next` (if non-null) for later, link `current` to the child, clear the child pointer, and fix `previous / next` doubly-linked wiring. When `current.next` is null but the stack is non-empty, pop and reattach.

```

class Solution {
    public Node flatten(Node head) {
        if (head == null) {
            return null;
        }
        Deque<Node> stack = new ArrayDeque<>();
        Node current = head;
        while (current != null) {
            if (current.child != null) {
                if (current.next != null) {
                    stack.push(current.next);    // defer the original continuation
                }
                current.next = current.child;
                current.next.prev = current;
                current.child = null;            // ← VARIATION: clear child after splicing
            }
            if (current.next == null && !stack.isEmpty()) {
                Node resumed = stack.pop();
                current.next = resumed;
                resumed.prev = current;
            }
            current = current.next;
        }
        return head;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#445 Add Two Numbers II

Description: Two numbers are stored as linked lists in big-endian order (most significant digit first). Add them and return the sum as a linked list, also in big-endian order.

Intuition: Addition needs least-significant digits first, but reversing the inputs is discouraged here. Push all digits onto two stacks; popping yields least-significant-first. Build the result by prepending nodes so the final list is big-endian.

Algorithm: Push every digit of each list onto its own stack. Pop both stacks while either is non-empty or a carry remains, summing with carry. Prepend each new digit node to the front of the result so order is correct without a final reversal.

```

class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        Deque<Integer> stack1 = new ArrayDeque<>();
        Deque<Integer> stack2 = new ArrayDeque<>();
        for (ListNode current = l1; current != null; current = current.next) {
            stack1.push(current.val);
        }
        for (ListNode current = l2; current != null; current = current.next) {
            stack2.push(current.val);
        }
        ListNode result = null;
        int carry = 0;
        while (!stack1.isEmpty() || !stack2.isEmpty() || carry != 0) {
            int sum = carry;
            if (!stack1.isEmpty()) {
                sum += stack1.pop();
            }
            if (!stack2.isEmpty()) {
                sum += stack2.pop();
            }
            carry = sum / 10;
            ListNode node = new ListNode(sum % 10);
            node.next = result;          // ← VARIATION: prepend to keep big-endian order
            result = node;
        }
        return result;
    }
}

```

Time $O(m + n)$ | **Space** $O(m + n)$

#708 Insert into a Sorted Circular Linked List

Description: Insert a value into a sorted circular linked list and return a reference to any node in the list. The given pointer may reference any node, or be null for an empty list.

Intuition: Walk one full loop looking for the correct gap between a node and its successor. Three cases cover it: a normal in-between slot, the wrap-around boundary (max→min) where the value is an extreme, and a uniform-value list where any spot works.

Algorithm: If the list is empty, create a self-referential node. Otherwise scan `previous / current` around the ring. Insert when `previous.val <= val <= current.val`, or at the wrap point `previous.val > current.val` when `val` is `>= max` or `<= min`. If a full loop completes with no fit (all equal), insert anywhere.

```

class Solution {
    public Node insert(Node head, int insertVal) {
        Node node = new Node(insertVal);
        if (head == null) {
            node.next = node;          // single-element circular list
            return node;
        }
        Node previous = head, current = head.next;
        while (true) {
            if (previous.val <= insertVal && insertVal <= current.val) {
                break;                // normal in-between slot
            }
            if (previous.val > current.val
                && (insertVal >= previous.val || insertVal <= current.val)) {
                break;                // ← VARIATION: wrap-around boundary
            }
            previous = current;
            current = current.next;
            if (previous == head) {
                break;                // full loop: all values equal
            }
        }
        previous.next = node;
        node.next = current;
        return head;
    }
}

```

Time $O(n)$ | **Space** $O(1)$

#725 Split Linked List in Parts

Description: Split a linked list into k consecutive parts as equal in size as possible. Earlier parts have sizes greater than or equal to later parts; some trailing parts may be empty.

Intuition: With length n , each part gets n / k nodes, and the first $n \% k$ parts get one extra. Walk through cutting off each part at its computed size.

Algorithm: Compute the length, then $\text{baseSize} = n / k$ and $\text{remainder} = n \% k$. For each of the k parts, the size is $\text{baseSize} + (i < \text{remainder} ? 1 : 0)$. Advance that many nodes, then sever the link to start the next part.

```

class Solution {
    public ListNode[] splitListToParts(ListNode head, int k) {
        int length = 0;
        for (ListNode current = head; current != null; current = current.next) {
            length++;
        }
        int baseSize = length / k;
        int remainder = length % k;          // ← VARIATION: first `remainder` parts get +1
        ListNode[] result = new ListNode[k];
        ListNode current = head;
        for (int i = 0; i < k; i++) {
            result[i] = current;
            int partSize = baseSize + (i < remainder ? 1 : 0);
            for (int j = 0; j < partSize - 1; j++) {
                current = current.next;
            }
            if (current != null) {
                ListNode next = current.next;
                current.next = null;          // sever this part
                current = next;
            }
        }
        return result;
    }
}

```

Time $O(n)$ | **Space** $O(k)$

#1171 Remove Zero Sum Consecutive Nodes from Linked List

Description: Repeatedly remove consecutive sequences of nodes that sum to zero until no such sequence remains. Return the resulting list.

Intuition: Use prefix sums. If two nodes share the same running prefix sum, every node strictly between them sums to zero and can be dropped. A hash map from prefix sum to node lets you splice out those runs in one pass.

Algorithm: First pass: compute prefix sums over a sentinel-prefixed list, storing the last node seen for each prefix sum in a map (later occurrences overwrite earlier ones). Second pass: for each prefix sum, jump directly to the last node holding that same sum, skipping the zero-sum span.

```

class Solution {
    public ListNode removeZeroSumSublists(ListNode head) {
        ListNode sentinel = new ListNode(0);
        sentinel.next = head;
        Map<Integer, ListNode> lastSeen = new HashMap<>();
        int prefix = 0;
        for (ListNode current = sentinel; current != null; current = current.next) {
            prefix += current.val;
            lastSeen.put(prefix, current);          // ← VARIATION: keep the latest node per sum
        }
        prefix = 0;
        for (ListNode current = sentinel; current != null; current = current.next) {
            prefix += current.val;
            current.next = lastSeen.get(prefix).next;  // skip any zero-sum span
        }
        return sentinel.next;
    }
}

```

Time $O(n)$ | **Space** $O(n)$

#1265 Print Immutable Linked List in Reverse

Description: Print all values of an immutable linked list in reverse, using only the exposed `getNext()` and `printValue()` API, without modifying the list.

Intuition: Recursion naturally reverses order — recurse to the end first, then print on the way back up the call stack.

Algorithm: If the node is null, return. Otherwise recurse on `head.getNext()` first, then call `head.printValue()`. Values print from tail to head as the stack unwinds.

```
class Solution {
    public void printLinkedListInReverse(ImmutableListNode head) {
        if (head == null) {
            return;
        }
        printLinkedListInReverse(head.getNext()); // ← VARIATION: recurse first, print on unwind
        head.printValue();
    }
}
```

Time $O(n)$ | **Space** $O(n)$